

[DOC] Fbfbfb

Documentación Técnica · AutoDocs AI · 07/07/2026

21	42	0	3
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 71/100

Documentación del Proyecto

Generado automáticamente por **AutoDocs AI** — 2026-07-07 04:11 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **JavaScript** y cuenta con 21 archivos fuente, 42 funciones, 0 clases y 3 endpoints de API documentados. El score de calidad del código es **71/100**.

Stack Tecnológico

Lenguaje	Cantidad
SQL	1 archivos
JSON	6 archivos
JavaScript	14 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	21
Funciones	42
Clases	0
Endpoints API	3
Score de calidad	71/100

Funciones documentadas	0 / 42
Clases documentadas	0 / 0

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/  
  (raíz)/  
  backend/  
  mobile/
```

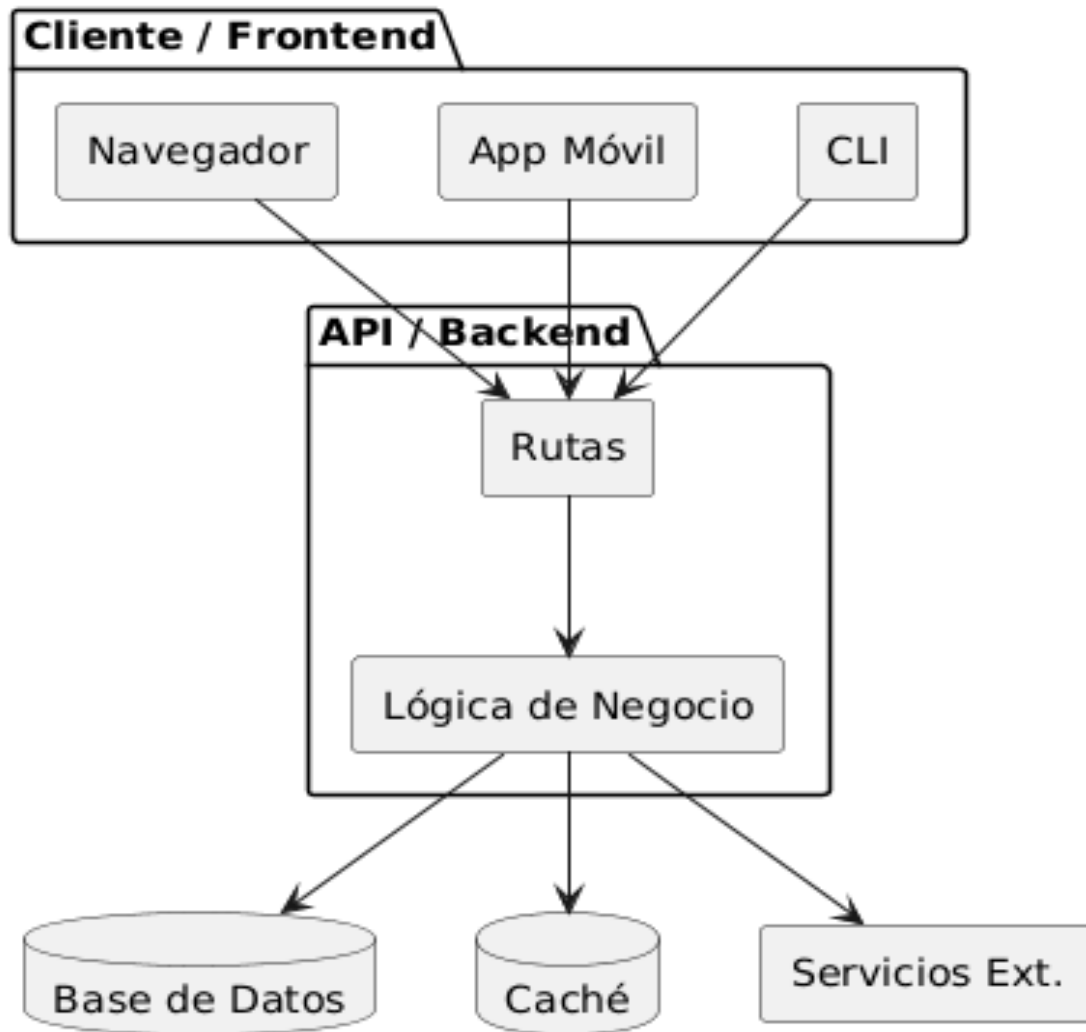
2.2 Distribución por Capas

- (raíz)/ — 1 archivos [.gitignore(1)]
- backend/ — 9 archivos [.example(1), .sql(1), .json(2), .sin ext(1), .js(3), .env(1)]
- mobile/ — 15 archivos [.js(11), .json(4)]

2.3 Distribución de Lenguajes

- JavaScript: 14 archivos (67%)
- JSON: 6 archivos (29%)
- SQL: 1 archivos (5%)

2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso

2.6 Diagrama de Clases

2.7 Patrones Detectados

- **API REST**: 3 endpoints detectados (Express.js)
- **Testing**: 1 funciones de prueba detectadas
- **Programación Asíncrona**: 12 funciones async/await

2.8 Insights de Arquitectura (IA)

Insights Arquitectónicos

1. **Patrones arquitectónicos probables**: Debido a la presencia de endpoints y la ausencia de clases, es probable que se esté utilizando un patrón arquitectónico basado en funciones, como un Monolito o un

enfoque de Servidor de Funciones (FaaS).

2. **Buenas prácticas detectadas:** El uso de JSON como formato de intercambio de datos es una buena práctica común en proyectos de software.

3. **Áreas de mejora sugeridas:** La falta de clases y la dependencia en funciones pueden indicar una arquitectura poco modular y difícil de mantener. Se sugiere considerar la implementación de patrones de diseño orientados a objetos.

4. **Complejidad del proyecto:** La complejidad parece ser moderada, con 42 funciones y 3 endpoints.

5. **Recomendaciones de escalabilidad:** Para mejorar la escalabilidad, se sugiere considerar la implementación de un patrón de arquitectura de Microservicios o la utilización de un framework que permita una mayor modularidad y reutilización de código.

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate   # Windows

# 3. Instalar dependencias
npm install

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: 3

Módulo: `backend/server.js`

[GET] `GET /health`

- **Framework:** Express.js

- **Archivo:** `backend/server.js`

Descripción del Endpoint `/health`

1. Propósito del endpoint:

El propósito de este endpoint es verificar si el servidor que lo ejecuta está funcionando correctamente. Piensa en ello como un "pulso" que se le toma al servidor para asegurarse de que esté vivo y listo para recibir solicitudes.

2. Funcionalidad:

Cuando se accede a este endpoint, el servidor realiza una serie de comprobaciones internas para asegurarse de que todos los componentes críticos estén funcionando como se espera. Es como hacer una revisión rápida de un vehículo antes de salir a la carretera para asegurarse de que todo esté en orden.

3. Información necesaria:

Para que este endpoint funcione, no se necesita proporcionar ninguna información adicional. Simplemente se accede a la ruta `/health` y el servidor se encarga del resto. Es como visitar una página web; solo necesitas saber la dirección (la ruta) para llegar allí.

4. Resultado:

El resultado de acceder a este endpoint es una respuesta que indica si el servidor está funcionando correctamente o no. Si todo está bien, el servidor devuelve un mensaje que dice algo así como "Estoy funcionando correctamente" o un código que indica éxito. Si hay algún problema, el servidor puede devolver un mensaje de error o un código que

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
- `400` — Solicitud incorrecta
- `401` — No autorizado
- `404` — Recurso no encontrado
- `500` — Error interno del servidor

[POST] `POST /rooms`

- **Framework:** Express.js
- **Archivo:** `backend/server.js`

Descripción del Endpoint de API: `/rooms`

1. Propósito del Endpoint

El propósito de este endpoint es permitir a los usuarios crear nuevas "salas" o "espacios" dentro de una aplicación o plataforma. Piensa en una sala como un lugar donde las personas pueden reunirse, charlar o compartir información.

2. Funcionalidad

Cuando se utiliza este endpoint, se envía una solicitud a la aplicación para crear una nueva sala. La aplicación then procesa esta solicitud y crea la sala con la información proporcionada. Es como cuando creas un nuevo grupo en una aplicación de mensajería para hablar con tus amigos o familiares.

3. Información Necesaria

Para que este endpoint funcione, necesitas proporcionar algunos datos básicos sobre la sala que deseas crear. Esto puede incluir:

- El nombre de la sala
- Una descripción breve de la sala
- Quizás algunos ajustes de privacidad o configuraciones específicas

Es como cuando creas un nuevo evento en una red social y necesitas darle un nombre, describir qué es y decidir quién puede asistir.

4. Resultado

Una vez que se crea la sala, el endpoint devuelve una respuesta que confirma que la sala ha sido creada con éxito. Esta respuesta puede incluir detalles como el ID único de la sala, su nombre y cualquier otra información relevante

Parámetros esperados:

- `body` (JSON, requerido): Cuerpo de la solicitud
- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
- `400` — Solicitud incorrecta
- `401` — No autorizado
- `404` — Recurso no encontrado
- `500` — Error interno del servidor

[POST] `POST /rooms/:code/join`

- **Framework:** Express.js
- **Archivo:** `backend/server.js`

Descripción del Endpoint

Propósito: El propósito de este endpoint es permitir que los usuarios se unan a una sala de reuniones o conferencia en línea.

Funcionalidad: Este endpoint se utiliza para conectar a un usuario a una sala de reuniones específica. Cuando se envía una solicitud al servidor, el endpoint verifica si la sala existe y si el usuario tiene permiso para unirse. Si todo está en orden, el usuario se une a la sala y puede interactuar con los demás

participantes.

Información necesaria: Para que este endpoint funcione, se necesita proporcionar el código de la sala de reuniones a la que se desea unir. Este código es como una llave que abre la puerta a la sala. Además, es posible que se requiera información adicional, como el nombre del usuario o una contraseña, para garantizar que solo las personas autorizadas puedan unirse a la sala.

Resultado: Cuando se une a una sala, el servidor devuelve una respuesta que indica si la operación fue exitosa o no. Si todo salió bien, el usuario podrá ver y interactuar con los demás participantes en la sala. Si hubo un problema, el servidor puede devolver un mensaje de error que explique qué salió mal.

Uso en la vida real: En la vida real, este endpoint se puede utilizar en aplicaciones de videoconferencia,

Parámetros esperados:

- **body** (JSON, requerido): Cuerpo de la solicitud
- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
- 400 — Solicitud incorrecta
- 401 — No autorizado
- 404 — Recurso no encontrado
- 500 — Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Total de funciones: 42

- [OK] Documentadas: 0 (0%)
- [X] Sin documentar: 42 (100%)

Visualización de documentación:



Funciones que requieren documentación:

- `generateRoomCode()` en `backend/server.js`
- `getRoomByCode(code)` en `backend/server.js`
- `getUsedWords(roomId)` en `backend/server.js`
- `maskWord(word, guessedLetters)` en `backend/server.js`
- `checkWin(word, guessedLetters)` en `backend/server.js`
- ... y 37 más

3.4 Planes de Prueba (Testing)

Se detectaron **1 funciones de prueba**:

Función de Test	Archivo
`testConnection`	`backend/test.js`

4. Clases

No se detectaron clases en este proyecto.

5. Documentación de Funciones

Total: **42 funciones**

Módulo: `backend/server.js`

`generateRoomCode`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "generateRoomCode" crea un código único para una sala virtual.

Necesita información para generar el código.

El resultado es un código que identifica la sala.

En la vida real, se usa para crear reuniones o salas de juego en línea.

Es importante porque permite a los usuarios unirse a la sala correcta de manera segura.

`async getRoomByCode`

- Parámetros: `code`
- Async: [ASYNC]
- Complejidad: [LOW] 1
- Documentada: [X]

La función "getRoomByCode" busca una sala en un sistema según un código único. Para funcionar, necesita saber el código de la sala. Luego, encuentra y devuelve la información de esa sala. En la vida real, se usa para identificar y acceder a salas virtuales o físicas, como en plataformas de reuniones en línea o sistemas de gestión de espacios. Esta función es importante porque permite a los usuarios encontrar y unir salas de manera rápida y precisa, lo que facilita la comunicación y la organización en el sistema.

`async getUsedWords`

- Parámetros: `roomId`
- Async: [ASYNC]
- Complejidad: [LOW] 1
- Documentada: [X]

La función "getUsedWords" ayuda a obtener las palabras utilizadas en una habitación específica. Necesita saber el identificador de la habitación (`roomId`) para funcionar. Una vez que lo recibe, devuelve una lista de palabras que se han utilizado en esa habitación. En la vida real, se puede utilizar para analizar conversaciones en chats o foros en línea. Es importante porque permite entender qué temas se discuten en una habitación, ayudando a moderar o mejorar la experiencia del usuario. En resumen, esta función es clave para el análisis de contenido en plataformas de comunicación en línea.

`maskWord`

- Parámetros: `word`, `guessedLetters`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "maskWord" sirve para ocultar letras de una palabra que no han sido adivinadas.

Necesita saber la palabra original y qué letras han sido adivinadas.

Produce una versión de la palabra con las letras no adivinadas ocultas.

Se usa en juegos de adivinanza de palabras, como "Ahorcado", para mostrar el progreso del jugador.

Es importante porque permite al jugador ver qué letras ya han sido encontradas y qué letras faltan por descubrir, haciendo el juego más emocionante y desafiante.

`checkWin`

- Parámetros: `word`, `guessedLetters`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "checkWin" determina si se ha ganado un juego. Necesita saber la palabra secreta ("`word`") y las letras adivinadas ("`guessedLetters`"). Devuelve un resultado que indica si se ha ganado o no. En la vida real, se usa en juegos de adivinanza de palabras, como el ahorcado. Esta función es crucial para determinar el fin del juego y anunciar al ganador. Su importancia radica en proporcionar un resultado justo y preciso, asegurando la integridad del juego.

`serializeRoom`

- Parámetros: `room`
- Async: —
- Complejidad: [LOW] 1

- Documentada: [X]

`async startRound`

- Parámetros: `roomCode`
 - Async: [ASYNC]
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

Módulo: `backend/words.js`

`isValidWord`

- Parámetros: `word`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`normalizeWord`

- Parámetros: `word`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`async fetchFromPalabrasDev`

- Parámetros: —
- Async: [ASYNC]
- Complejidad: [LOW] 1
- Documentada: [X]

`async fetchBatchFromPalabrasDev`

- Parámetros: `count = 5`
- Async: [ASYNC]
- Complejidad: [LOW] 1
- Documentada: [X]

`async getRandomWord`

- Parámetros: `usedWords = []`
- Async: [ASYNC]
- Complejidad: [LOW] 1
- Documentada: [X]

Módulo: `backend/test.js`

`async testConnection`

- Parámetros: —
 - Async: [ASYNC]
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

Módulo: `mobile/App.js`

`App`

- Parámetros: —
 - Async: —
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

Módulo: `mobile/components/HangmanDrawing.js`

`HangmanDrawing`

- Parámetros: `{ wrongAttempts, size = 200 }`
 - Async: —
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

Módulo: `mobile/components/Keyboard.js`

`Keyboard`

- Parámetros: `{ guessedLetters, onGuess, disabled, currentWord }`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`getLetterStatus`

- Parámetros: `letter`
- Async: —
- Complejidad: [LOW] 1

- Documentada: [X]

`renderKey`

- Parámetros: `letter`
 - Async: —
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

Módulo: `mobile/app/GameScreen.js`

`useSounds`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`buildInitialSPState`

- Parámetros: `{ word, playerName, difficulty, hintsCount, playerWins }`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`GameScreen`

- Parámetros: `{ route, navigation }`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`clearTimerInterval`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`resetTimer`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

startTimer

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

StatCell

- Parámetros: { icon, color, label, value }
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

shakeScreen

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

bounceWin

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

flashLoss

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

pulseLetter

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

handleHint

- Parámetros: —
- Async: —

- Complejidad: [LOW] 1
- Documentada: [X]

`processGuess`

- Parámetros: `letter`, `isHint = false`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`handleGuess`

- Parámetros: `letter`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`handleExit`

- Parámetros: `confirmed = false`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`async handleNewGame`

- Parámetros: —
- Async: [ASYNC]
- Complejidad: [LOW] 1
- Documentada: [X]

Módulo: `mobile/app/HomeScreen.js`

`HomeScreen`

- Parámetros: `{ route, navigation }`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`async createRoom`

- Parámetros: —
- Async: [ASYNC]
- Complejidad: [LOW] 1

- Documentada: [X]

`async joinRoom`

- Parámetros: —
 - Async: [ASYNC]
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

Módulo: `mobile/app/WaitingScreen.js`

`WaitingScreen`

- Parámetros: { `route`, `navigation` }
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`async shareCode`

- Parámetros: —
 - Async: [ASYNC]
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

Módulo: `mobile/data/words.js`

`async getRandomWord`

- Parámetros: `usedWords = []`
- Async: [ASYNC]
- Complejidad: [LOW] 1
- Documentada: [X]

`getRandomWordFallback`

- Parámetros: `usedWords = []`
 - Async: —
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

Módulo: `mobile/hooks/useSocket.js`

useSocket

- Parámetros: —
 - Async: —
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis del código, las entidades principales son:

- No se pudieron inferir entidades del código fuente.
-

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

- Lenguaje principal: JavaScript
- Archivos detectados: 21

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
npm install

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
npm start
```

7.3 Despliegue con Docker

```
FROM node:20-slim
```

```
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 8000
CMD ["node index.js"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

- ☐ Variables de entorno configuradas
- ☐ Base de datos migrada
- ☐ Tests ejecutados y pasando
- ☐ Logs configurados
- ☐ HTTPS habilitado en producción
- ☐ Backups programados
- ☐ Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-07-07

- Versión inicial analizada por AutoDocs AI
 - Documentación técnica generada automáticamente
 - Análisis de calidad del código completado
-

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 71/100 (Bueno [YELLOW])

Distribución de Documentación:

- Funciones documentadas: 0/42 (0%)
- Clases documentadas: 0/0

Análisis de Complejidad:

- Complejidad promedio: 1.0
- Complejidad máxima: 1
- Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: ↓ Baja

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

- Agregar docstrings a las 42 funciones sin documentar

8.4 Errores de Análisis

Sin errores detectados. [OK]

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

• Funciones sin documentar: 42

- generateRoomCode en backend/server.js
- getRoomByCode en backend/server.js
- getUsedWords en backend/server.js
- maskWord en backend/server.js
- checkWin en backend/server.js
- ... y 37 más

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
■ Alta	> 5	0	Requieren refactorización urgente
■ Media	4–5	0	Candidatas a simplificación
■ Baja	≤ 3	42	Complejidad aceptable

■ No se detectaron funciones con complejidad ciclomática alta. El código mantiene un buen nivel de simplicidad.

B) Funciones Sin Documentación

42 de 42 funciones carecen de docstring (100%).

Función	Archivo	Línea	Complejidad
`generateRoomCode`	`backend/server.js`	?	1
`getRoomByCode`	`backend/server.js`	?	1
`getUsedWords`	`backend/server.js`	?	1
`maskWord`	`backend/server.js`	?	1
`checkWin`	`backend/server.js`	?	1
`serializeRoom`	`backend/server.js`	?	1
`startRound`	`backend/server.js`	?	1
`isValidWord`	`backend/words.js`	?	1

`normalizeWord`	`backend/words.js`	?	1
`fetchFromPalabrasDev`	`backend/words.js`	?	1
`fetchBatchFromPalabrasDev`	`backend/words.js`	?	1
`getRandomWord`	`backend/words.js`	?	1
`testConnection`	`backend/test.js`	?	1
`App`	`mobile/App.js`	?	1
`HangmanDrawing`	`mobile/components/HangmanDrawing.js`	?	1
`Keyboard`	`mobile/components/Keyboard.js`	?	1
`getLetterStatus`	`mobile/components/Keyboard.js`	?	1
`renderKey`	`mobile/components/Keyboard.js`	?	1
`useSounds`	`mobile/app/GameScreen.js`	?	1
`buildInitialSPState`	`mobile/app/GameScreen.js`	?	1

... y 22 funciones más sin documentar.

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

Se detectaron **1 grupo de funciones con nombres similares** repartidas en distintos archivos:

Patrón: `_getrandomword / getrandomword_` — aparece en 2 archivos distintos

Función	Archivo	Línea	Complejidad
`getRandomWord`	`backend/words.js`	?	1
`getRandomWord`	`mobile/data/words.js`	?	1

■ **Sugerencia:** Las funciones `getRandomWord` y `getRandomWord` realizan operaciones similares (`getrandomword`) en módulos distintos. Considera crear una función compartida en un módulo utilitario (por ejemplo `utils/getrandomword.py` o `helpers/common.py`) que reciba el contexto como parámetro, y llamarla desde ambos archivos (`backend/words.js`, `mobile/data/words.js`).

D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`mobile/app/GameScreen.js`	16	1.0	0%	■ Revisar
`backend/server.js`	7	1.0	0%	■ OK
`backend/words.js`	5	1.0	0%	■ OK

`mobile/components/Keyboard.js`	3	1.0	0%	■ OK
`mobile/app/HomeScreen.js`	3	1.0	0%	■ OK
`mobile/app/WaitingScreen.js`	2	1.0	0%	■ OK
`mobile/data/words.js`	2	1.0	0%	■ OK
`backend/test.js`	1	1.0	0%	■ OK
`mobile/App.js`	1	1.0	0%	■ OK
`mobile/components/HangmanDrawing.js`	1	1.0	0%	■ OK
`mobile/hooks/useSocket.js`	1	1.0	0%	■ OK

■ `mobile/app/GameScreen.js` tiene 16 funciones. Considera dividirlo en submódulos más específicos siguiendo el principio de responsabilidad única (SRP).

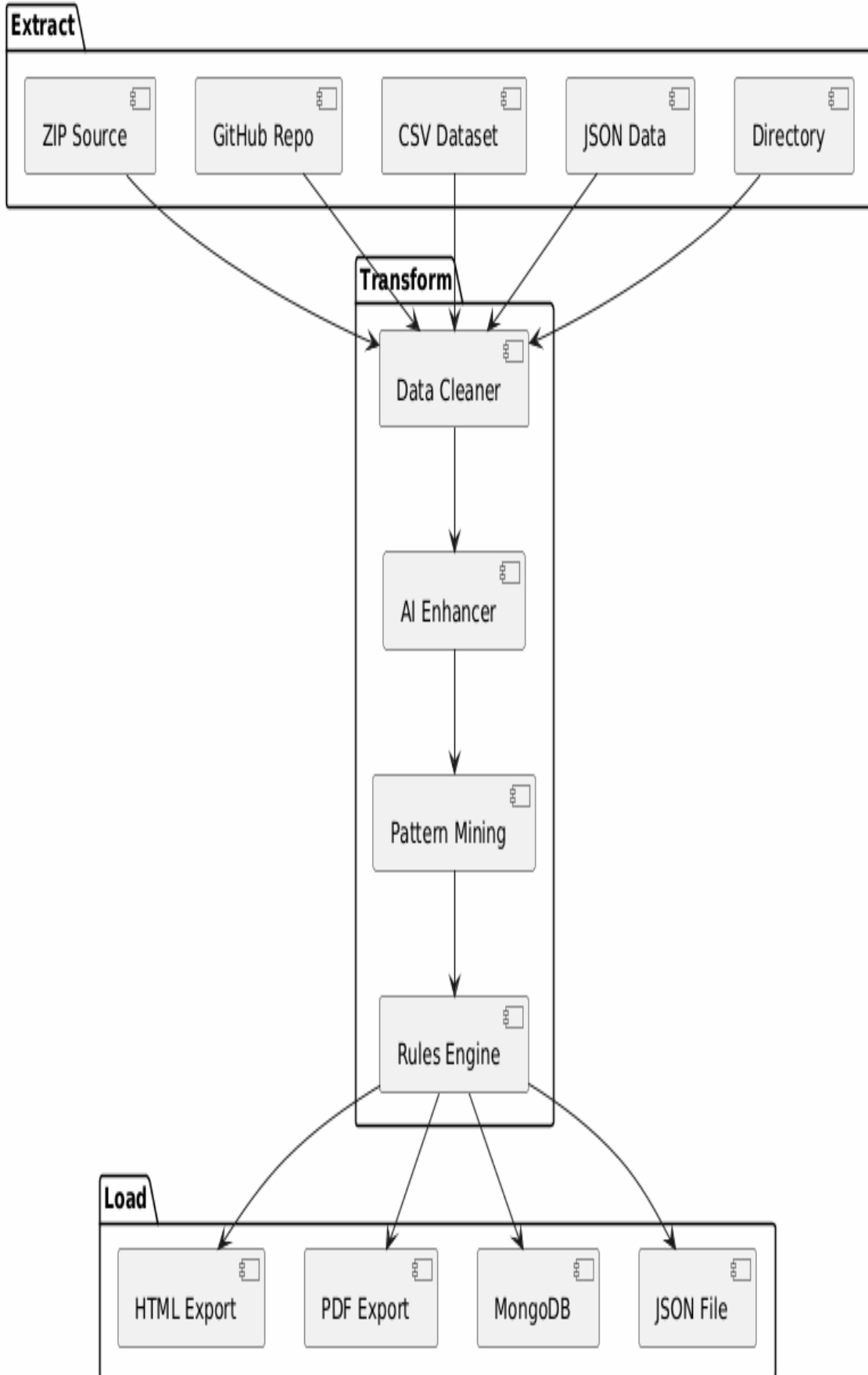
E) Patrones Frecuentes Adicionales (Análisis IA)

- **get:** 5 ocurrencias
 - Ejemplos: ['getRoomByCode', 'getUsedWords', 'getLetterStatus']
- **fetch:** 2 ocurrencias
 - Ejemplos: ['fetchFromPalabrasDev', 'fetchBatchFromPalabrasDev']

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):

Pipeline ETL - AutoDocs AI



Fases del Pipeline:

- **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales
- **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones
- **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-07-07 04:11 UTC

Generado automáticamente por AutoDocs AI